ZCAM LATAM Hub — Secure Azure Databricks Platform

Technical design proposal | System Administrator assignment | Version 0.1
Status: Proposed design, not deployed. **Scope:** Production and development Databricks environments, shared enterprise connectivity, controlled data access, secrets and resilience.

1. Executive summary and assumptions

This design extends the existing Hub VNet, Azure Firewall, VPN Gateway, Microsoft Entra ID tenant, Azure DevOps platform and production Azure SQL Database rather than replacing them. A separate Azure Databricks Premium workspace is proposed for each of the existing production and development subscriptions. Each uses customer-managed VNet injection, Secure Cluster Connectivity, private endpoints and a least-privilege access model. A secondary recovery foundation is planned for production.

Assumptions to validate with ZCAM: (A1) Primary region is Brazil South and there is a secondary region approved by Legal, Security and Data Governance; no cross-border PII replication occurs until approved. (A2) The 15 Data Engineers are managed through Entra ID groups. (A3) The two external HTTPS APIs are ingestion sources only: no customer PII is transmitted to the APIs. (A4) Source SQL ownership and any cross-region SQL replication are controlled by another team. (A5) Compute use and data volumes are bounded as shown in the cost model. (A6) Classic VNet-injected compute is used initially; serverless connectivity is a separately assessed phase. (A7) Three named storage accounts in the brief are production resources; development uses different accounts with synthetic or de-identified data.

2. Network design and private connectivity

The current Hub VNet retains the existing Azure Firewall and VPN Gateway. Production and development spokes are isolated in separate subscriptions with independent resource groups, VNets, Databricks workspaces, storage resources, Key Vaults and roles. Hub-to-spoke peering provides controlled connectivity; there is no direct development-to-production routing. Example, non-overlapping CIDRs are `10.20.0.0/16` (production) and `10.30.0.0/16` (development), subject to enterprise IPAM approval.

Each Databricks workspace uses two dedicated, NSG-attached subnets delegated to `Microsoft.Databricks/worksp` a host subnet and a container subnet, provisionally `/24` each. The "public subnet" field used by the provider is a Databricks historical name; with `no_public_ip = true`, it does **not** imply Internet-facing cluster IPs. A separate subnet holds private endpoints; another optional subnet is reserved for trusted operational agents. NSG and UDR rules must preserve Databricks-required platform connectivity and are validated before deployment. The Azure Firewall is the approved outbound path, with the two API FQDNs restricted to TCP 443; required Databricks platform endpoints are separately allowlisted. Because private VNets no longer automatically provide Internet egress, this explicit path is a deployment prerequisite.

Full private workspace isolation is the target: public workspace access disabled; `databricks_ui_api` front-end and back-end Private Link endpoints, plus regional browser authentication/private DNS arrangements. Users reach the UI from the São Paulo office over the VPN. Private DNS zones are linked to the relevant transit/spoke VNets and resolved consistently through the Hub DNS architecture. The existing SQL logical server is accessed through a production private endpoint, with the SQL owner's approval; its public access must be disabled after private connectivity is validated.

All three production ADLS Gen2 accounts use public network access disabled. **Both `dfs` and `blob` private endpoints** are planned where required for hierarchical namespace operations, with `privatelink.dfs.core.windows.net` and `privatelink.blob.core.windows.net` zones. A separate

Key Vault private endpoint uses `privatelink.vaultcore.azure.net`. The SQL private DNS zone is `privatelink.database.windows.net`. No notebook, API pipeline or BI workload may transfer PII over an Internet-facing route. DNS, routing, and negative tests must prove this behavior.

Data flows: External API 1 and API 2 → approved HTTPS egress through Hub Firewall → authenticated ingestion process → `stlatamraw` → Databricks classic compute → `stlatamcurated` (PII, restricted) → `stlatamanalytics` (approved aggregates). The production Databricks environment also queries the existing Azure SQL privately. Development uses equivalent **separate** storage resources containing only synthetic or irreversibly de-identified test data and cannot query production PII or SQL by default.

3. Identity, RBAC and data governance

Entra ID is the workforce identity source, with MFA and Conditional Access where corporate policy requires. Role assignments should be granted to groups, not individuals, with periodic access reviews. A suggested group design is: `ZCAM-DBX-PROD-ADMINS` (very small set of workspace administrators), `ZCAM-DBX-PROD-ENGINEERS` (job/notebook access without infrastructure administration), `ZCAM-DBX-DEV-ENGINEERS` (development access only), `ZCAM-DATA-PII-READERS` (restricted and approved PII access), and `ZCAM-PLATFORM-OPERATORS` (monitoring/incident roles).

Azure RBAC examples: Subscription Reader for auditors where authorized; Resource Group Contributor for tightly controlled deployment identities **only where necessary**; Network Contributor scoped to the approved spoke VNet for the provisioning principal; Key Vault Secrets User scoped to the relevant vault for approved runtime identities; Storage Blob Data Reader/Contributor scoped to the correct storage account/container for trusted access identities. Avoid Owner, broad Contributor and shared storage account keys. Separate resource-control RBAC from Unity Catalog data privileges: Unity Catalog grants SELECT, USE CATALOG/SCHEMA and write permissions only at the appropriate object scope, using managed identities/access connectors and storage credentials. PII access should be audited; development groups are excluded.

Azure DevOps uses protected environments, review approvals, branch protection, a private build agent with network access, and workload identity federation rather than stored service-principal passwords. The submitted Terraform deliberately defines **one** reusable module (Databricks workspace with VNet injection), called once per subscription; network and data prerequisites are dependencies managed outside that module's scope.

4. Key Vault strategy

Deploy separate production and development Key Vaults. Use Azure RBAC, private endpoints, public network access disabled, private DNS, soft delete, purge protection, diagnostic logs and least privilege. Secrets include the two external API credentials and any legacy connection secrets that cannot be replaced by Entra authentication. Prefer workload identities, short-lived credentials and managed identity for ADLS and SQL whenever supported; identity-based access reduces the number of secrets needing storage.

Secrets must not appear in notebooks, Git, Terraform variable files, pipeline YAML, logs or Terraform outputs. A VNet-connected ingestion/orchestration workload with an approved workload identity retrieves API credentials from the private Key Vault at runtime and invokes the two public HTTPS APIs through the Firewall. Private Azure DevOps agents can access the Key Vault through the private endpoint as needed for deployment, but do not print secret values. Configure rotation with ownership, expiration monitoring, alerts, dual-credential overlap when supported, and an operational runbook; use a candidate **90-day maximum review/rotation window**, adjusted to each provider's policy and capability. A failure or expiry alert must reach the on-call team before the credential expires.

Important compatibility choice: Databricks Azure Key Vault-backed secret scopes currently require the Key Vault **access-policy** permission model and specific network exceptions, not a fully RBAC-only/private-endpoint-only vault. Therefore, this proposal does **not** assume those scopes work with the chosen strict private Key Vault posture. It uses identity-based access and a VNet-connected secret-consuming workload instead. A scope-based alternative would require a separately documented security exception and design review.

5. Monthly cost planning (USD, illustrative)

This is an **allowance-based scenario, not an Azure quote**. Databricks pricing depends on DBUs, VM SKUs, region, performance tier, concurrency and runtime. Validate actual rates in Azure Pricing Calculator and with the enterprise agreement before the design is approved.

Service / workload	Assumption	Monthly allowance (USD)
Databricks production compute + DBUs	8 h/day × 22 days × $16 blended/h	2,816
Databricks development compute + DBUs	4 h/day × 20 days × $8 blended/h	640
Three production + separate development storage accounts	Capacity and operations allowance	600
Production DR preparation, replication and backups	Secondary foundation allowance	800
Networking: PE, DNS, data processing, allocated shared Firewall	Incremental / allocated cost	900
Azure Monitor, Log Analytics and alerting	Controlled retention	300
Key Vault	Operations and secrets allowance	50
API and data-transfer allowance	Usage constrained by routing and job schedules	200
Subtotal		**6,306**
Contingency (20%)		**1,261**
Scenario total	**Under $8,000 target if assumptions hold**	**7,567**

Cost boundaries: Existing Azure SQL baseline charges and historical shared Hub fixed costs are assumed sunk/allocated outside the net-new build, except for the $900 network allocation above. If Finance defines the $8,000 cap as inclusive of all existing services or if storage, DBUs, cross-region transfer or private networking exceed the allowances, rebaseline and right-size before approval. Enforce per-environment tags, Azure budgets (50/75/90/100% notifications), compute policies, automatic termination, scheduled dev use, job clusters instead of continuously running interactive compute, and monthly cost reviews. A 20% contingency is not a guarantee.

6. Production disaster recovery: RPO 4h / RTO 2h

Targets are **unverified objectives**. Propose a warm, security-approved secondary-region foundation: deployment templates, private DNS, connectivity, workspace configurations, access groups, job defini-

tions, replicated storage and tested access to required source systems. Recovery runs from Azure DevOps with code/versioned assets. All customer PII replication requires explicit data residency approval. The managed Databricks DR service could be assessed separately but has gated availability, Mission Critical add-on requirements and additional cost; it is **not assumed** in the base estimate.

A controlled replication design should aim at a checkpoint frequency materially shorter than 4 hours (e.g., at most 2 hours), track last successful recovery-consistent Delta snapshot / table state, replicate Unity Catalog configuration and deployable assets, and alert on lag. Asynchronous replication alone does not establish guaranteed RPO; measure actual end-to-end data age, dependency availability and restore consistency. The existing Azure SQL Database is a critical dependency: the database owner must commit to geo-recovery/failover or an approved degraded-mode design. A healthy Databricks secondary workspace without accessible SQL may fail the business RTO.

Example timed drill: (1) detect and declare incident, freeze primary writes; (2) verify replica consistency and switch approved private DNS/routes; (3) activate secondary compute and deploy or enable jobs; (4) verify Unity Catalog access, SQL dependency, Key Vault secrets and protected data paths; (5) run reconciliation and smoke tests; (6) resume workloads and notify stakeholders. Measure elapsed time to business service restoration and latest usable data timestamp, not merely time to start a cluster. Run at least quarterly, document incidents and maintain a tested failback plan.

7. Trade-offs and delivery acceptance

1. **Strict Private Link** reduces accidental public exposure and helps satisfy the PII requirement, but increases private endpoint, DNS, support and operational complexity. Browser authentication and back-end control plane paths must be tested before changing production access.
2. **VNet-injected classic compute** enables predictable routing and firewall control. Serverless compute is deferred because its separate network connectivity design and costs must be proven.
3. **Two subscriptions with no dev-to-prod PII access** reduces blast radius and accidental disclosure, at the cost of duplicate workspace/storage controls.
4. **RBAC-only private Key Vault** is preferred over a convenience secret-scope integration that needs access policies / network exceptions. A VNet-connected consumer adds integration work but keeps the vault private.
5. **Warm DR** contains idle compute costs relative to active-active, but its two-hour target is conditional on automation, replication and successful timed drills.
6. **Existing Hub reuse** reduces incremental network cost but introduces dependence on central firewall throughput, DNS availability and change control.

Validation checklist: `terraform fmt` and `terraform validate`; peerings and NSGs checked; private DNS resolves to private IPs; public access to PII data and SQL denied; two external API destinations are the only approved non-platform HTTPS egress routes; non-PII dev-only data verified; Azure RBAC and Unity Catalog least privilege tested; Key Vault rotation alert tested; Azure DevOps access and state security reviewed; cost forecast reconciled with calculator; RPO/RTO demonstrated by a timed, signed-off drill.

Implementation note: The submission contains a **single Terraform module** for the VNet-injected Azure Databricks workspace. It does not claim to provision the entire network, Key Vault, storage, SQL, DR or CI/CD foundation. This is intentional to comply with the one-module constraint.

Official reference documentation

- Azure Databricks VNet injection: https://learn.microsoft.com/en-us/azure/databricks/security/network/classic/vn inject

- Azure Databricks Private Link: https://learn.microsoft.com/en-us/azure/databricks/security/network/concepts/pr
  link
- Azure Databricks secret scopes: https://learn.microsoft.com/en-us/azure/databricks/security/secrets/secret-
  scopes
- Azure Storage Private Endpoints: https://learn.microsoft.com/en-us/azure/storage/common/storage-
  private-endpoints
- Azure SQL Private Link: https://learn.microsoft.com/en-us/azure/azure-sql/database/private-
  endpoint-overview
- Azure Databricks DR: https://learn.microsoft.com/en-us/azure/databricks/admin/disaster-recovery
- Terraform azurerm_databricks_workspace: https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs